Technical Assessment

AI-First Full-Stack Engineer

Role	AI-First Full-Stack Engineer — sole technical hire
Issued by	Team Beauty & Cosmix / Cosmix Manufacturing
Duration	3–4 hours (do not exceed 5 hours)
Submission	GitHub repo link + short Loom walkthrough (5–10 min)
Deadline	Within 48 hours of receiving this document
Tools allowed	Claude, Cursor, Copilot, any AI tooling — encouraged

> *This test is designed to be completed faster with AI tools. We are not testing whether you can code from memory — we are testing whether you understand systems, make good architectural decisions, and can ship working software quickly. Show us how you work.*

What we are looking for in this test:

- Strong logic and systems thinking across interconnected problems
- Real working code — not pseudocode, not descriptions
- Clean, readable structure that a second developer could pick up
- Evidence that you use AI tooling as a force multiplier, not a crutch
- Good judgment about what to build vs. what to skip under time pressure

Overview

You are joining a multi-brand holding company in the beauty and cosmetics manufacturing space. The business operates Team Beauty and Cosmix — a private-labelling and contract manufacturing operation — alongside several consumer brand webstores built on Shopify.

Your role as the sole technical hire is to build and maintain the company's entire technology stack: AI agents, ERP integrations, customer-facing apps, data pipelines, and internal tools. This test covers four of those areas. You will not be expected to complete everything perfectly — prioritise correctness and clarity over quantity.

> *Before you write a single line of code, spend 10–15 minutes reading all four tasks end to end. The most successful submissions show evidence of thinking across tasks — shared database schemas, reusable API patterns, consistent data models.*

01 Database design & AI knowledge base

~ 60 min

Context

The business has multiple brands, each with their own products, customers, and orders. All brands share the same manufacturing operation. You need to design a database that serves as the foundation for all other systems in this test.

Task 1A — Schema design

Design a PostgreSQL schema that satisfies all of the following requirements:

- Multiple brands can exist (e.g. Team Beauty, Cosmix, future brands)
- Each brand has its own product catalogue — products have a name, SKU, category, and price
- Customers can exist across multiple brands using a single shared identity (same person, one record)
- Orders belong to a specific brand but are linked to the shared customer record
- Raw materials have a name, unit of measure, current stock quantity, reorder level, and cost per unit
- Formulations (product recipes) are linked to finished products and contain a list of raw materials with quantities

Deliverable: A .sql file with all CREATE TABLE statements, indexes, and foreign key constraints. Include a brief comment on any design decisions that were not obvious.

Task 1B — Vector knowledge base setup

The company's customer-facing AI agent needs to answer questions about packaging options, MOQs, lead times, and formulation capabilities. This information currently lives in an Excel file (you will receive a sample in the repo).

Write a Python script that:

1. Reads a CSV file (you may create a sample with 10–15 realistic rows of cosmetics product/packaging data)
2. Chunks the data into meaningful pieces (explain your chunking strategy in a comment)
3. Generates embeddings using the OpenAI embeddings API or any open-source alternative
4. Stores the embeddings in a local vector store (Chroma, FAISS, or Supabase pgvector — your choice)
5. Exposes a simple query function: given a natural language question, return the top 3 most relevant chunks

Deliverable: A working Python script. Include a sample .csv file. The query function must work when we run it.

> *Bonus: Add a basic CLI that lets us type a question and see the retrieved chunks printed to the terminal.*

02 AI customer intake agent (bilingual)

~ 75 min

Context

The company receives inbound enquiries from potential private-label clients via WhatsApp, Instagram, Facebook, email, and phone. These enquiries come in both English and Urdu. The first AI agent (Agent 01) needs to qualify these leads, gather key information, and book a meeting.

What to build

Build a FastAPI application that simulates this agent. You do not need to connect to real WhatsApp or Instagram APIs — simulate the channel with a simple REST endpoint that accepts a JSON payload with a message and channel field.

The agent must:

1. Detect the language of the incoming message (English or Urdu) and respond in the same language
2. Maintain conversation state across multiple messages for the same session (use a session_id in the payload)
3. Collect the following information through natural conversation: company name, contact name, product category of interest, target quantity (MOQ context), timeline, and brand goals
4. Use the vector knowledge base from Task 1B to answer specific questions about packaging options or MOQs
5. When all required fields are collected, return a structured JSON summary of the lead

API specification

Your API must expose at minimum:

- POST /chat — accepts { session_id, channel, message } — returns { reply, language_detected, fields_collected, complete }
- GET /lead/{session_id} — returns the structured lead summary once complete is true

Deliverable: Working FastAPI app. Include a README with curl examples for both an English and an Urdu conversation flow. The agent must be powered by the Claude API or OpenAI — not a rule-based chatbot.

You may use hardcoded Urdu prompts in your system prompt to handle the bilingual requirement. What matters is that the agent detects the language and switches correctly. Show us the prompt you used and explain your design choices in the README.

03 Price comparison scraper

~ 50 min

Context

The business wants to monitor competitor and supplier pricing for cosmetic raw materials and packaging. You need to build a scraper that collects product data, stores it, and surfaces price changes.

What to build

You may scrape any publicly accessible website that lists products with prices. Good candidates include Amazon product search results, Alibaba supplier listings, or any open cosmetics supplier site. Do not scrape sites that explicitly prohibit it in their robots.txt.

Build a Python scraper that:

1. Accepts a list of search terms (e.g. "glass dropper bottle 30ml", "aluminium cosmetic jar")
2. Scrapes product name, price, seller/supplier, URL, and date scraped for each result
3. Stores results in the PostgreSQL schema you designed in Task 1 (add a price_comparisons table)
4. Detects when a price has changed from the previous scrape and flags it
5. Can be run on a schedule (show how you would schedule it — cron, APScheduler, or similar)

Deliverable: Working Python scraper using Playwright or Scrapy. Must actually run and return real data. Include the target URL(s) you chose and why.

We will run your scraper during evaluation. If it fails or returns empty results, the task does not pass regardless of code quality. Test it before submitting.

04 Shopify app — product review widget

~ 45 min

Context

The company's Shopify stores need a review system. You will build a minimal but functional Shopify embedded app that lets customers submit reviews and displays them on the product page.

What to build

Build a Shopify app (you may use the Shopify CLI to scaffold it) that:

1. Embeds in the Shopify admin — merchants can see reviews submitted for their products
2. Provides a storefront theme extension (App Block) that displays reviews on the product page
3. Accepts review submissions via a simple form: customer name, rating (1–5), and review text
4. Stores reviews in your own database (connect to the PostgreSQL schema from Task 1 — add a reviews table linked to a product SKU)
5. Uses the Claude API to generate a one-sentence AI summary of all reviews for a product (e.g. "Customers love the texture but note the scent is strong")

Deliverable: A Shopify app that installs on a development store. Include a short README explaining how to install it. The App Block must render correctly on a product page.

> *You do not need to publish to the Shopify App Store. A working installation on a free Shopify Partner development store is sufficient. Focus on the review submission flow and the AI summary — that is what we will evaluate.*

Scoring guide

Total: 100 points. A score of 70+ is a pass. We evaluate correctness, clarity, and judgment — not perfection.

Task	Points	Pass criteria
Task 1A — Database schema	**/15**	Schema is normalised, constraints are correct, multi-brand shared customer identity is implemented
Task 1B — Vector knowledge base	**/15**	Script runs, embeddings are stored, query function returns relevant results
Task 2 — Bilingual AI agent	**/25**	API runs, language detection works, conversation state is maintained, lead summary is returned
Task 3 — Price scraper	**/20**	Scraper runs and returns real data, price change detection works, scheduling is demonstrated
Task 4 — Shopify review app	**/15**	App installs, review submission works, AI summary is generated and displayed
Cross-task consistency	**/5**	Shared database schema used across tasks, consistent data models, evidence of thinking across projects
Code quality & README	**/5**	Clear structure, comments on non-obvious decisions, README is usable by someone who has never seen the code
Total	**/100**	70+ = pass. Invite to technical interview.

Submission instructions

- Create a single public GitHub repository named teambeauty-assessment
- Structure your repo with one folder per task: /task1, /task2, /task3, /task4
- Each folder must contain a README.md with setup instructions and any design notes
- Include a top-level README.md that explains how you approached the test, which AI tools you used and how, and any tradeoffs you made under time pressure
- Record a 5–10 minute Loom walkthrough showing each task running. Narrate your thinking — we want to hear how you reason, not just what you built.
- Email the GitHub link and the Loom link to the address provided in your offer correspondence

> *We read the README and watch the Loom before we look at the code. A clear explanation of your tradeoffs is worth more than extra features. If you ran out of time on a task, say so and explain what you would have done.*

What separates good from great

Good submission:

- All four tasks work as specified
- Code is readable and structured
- README explains how to run each task

Great submission:

- The database schema from Task 1 is used consistently across all four tasks — one schema, four consumers
- The AI agent in Task 2 references real data from the vector store built in Task 1
- The scraper in Task 3 feeds into the same PostgreSQL schema, not a separate database
- The Shopify review app stores reviews in the same schema with a product SKU that could match a formulation
- The Loom walkthrough demonstrates that you understand the bigger picture — these are not four isolated exercises, they are one integrated system

> *The best submission we have ever seen was 60% complete but showed a deeply coherent system design and a Loom where the candidate explained exactly what they would have built with another 2 hours. We hired that person immediately.*

Questions & clarifications

If anything in this document is ambiguous, make a reasonable assumption, document it in your README, and proceed. Do not wait for clarification — time-boxing and decision-making under ambiguity is part of what we are evaluating.

For urgent technical blockers only (e.g. access issues), contact the email address in your offer correspondence.

Good luck. We look forward to seeing what you build.